

Overview:

The purpose of this lab is to explore a number of ways to capture, display, and process video files using Python. The lab also compares the CPU performance with the hardware accelerated GPU processing using OpenCV CUDA and PyCuda.

Exercises:

1) Local Capture

- a) Commands were used to verify the camera hardware is visible and working correctly. A baseline OpenCV capture script provided for the notebook is used to establish all hardware is working. Screenshot not provided as this was run locally, not remotely.

2) Camera Abstraction and Jetcam Utilities

- a) The goal of this exercise was to implement a threaded camera class for stable frame acquisition
- b) A camera class is made using traitlets and threading to allow for asynchronous frame capture, the notebook provides the USBCamera and CSICamera classes depending on the hardware setup.

```
class Camera(traitlets.HasTraits):
    value = traitlets.Any()
    width = traitlets.Integer(default_value=224)
    height = traitlets.Integer(default_value=224)
    format = traitlets.Unicode(default_value='bgr8')
    running = traitlets.Bool(default_value=False)

    def __init__(self, *args, **kwargs):
        super(Camera, self).__init__(*args, **kwargs)
        if self.format == 'bgr8':
            self.value = np.empty((self.height, self.width, 3), dtype=np.uint8)
            self._running = False

    def _read(self):
        raise NotImplementedError

    def read(self):
        if self._running:
            raise RuntimeError('Cannot read directly while camera is running')
        self.value = self._read()
        return self.value

    def _capture_frames(self):
        while True:
            if not self._running:
                break
            self.value = self._read()

    @traitlets.observe('running')
    def _on_running(self, change):
        if change['new'] and not change['old']:
            self._running = True
            self.thread = threading.Thread(target=self._capture_frames)
            self.thread.start()
        elif change['old'] and not change['new']:
            self._running = False
            self.thread.join()
```

3) Video file manipulation

- a) A sample video is input and displayed using ipywidgets



- b) Using the Canny Edge detection in the CV2 library, the input video is transformed



4) Low-Level Kernel Programming with PyCUDA

- a) The goal of this portion was to implement a BGR to Gray conversion using CUDA C

- b) The provided kernel manually calculates grayscale values for each pixel, similar to the previous lab converting images to grayscale.